

Primer Parcial de Programación Orientada a Objetos (72.33)

28/09/2023

Ejercicio 1 (3 puntos)

Se desean modelar los “*slots*” de reservas de atracciones de un parque de diversiones. Un slot es una franja de una cierta **cantidad de minutos**. Cuando los visitantes del parque quieran reservar una visita a la atracción deben indicar el slot correspondiente. Un slot se identifica por el **horario de inicio** de su franja.

Por ejemplo, si los slots de una atracción son de 30 minutos y el horario de apertura de la atracción es 11:00 y el de cierre es 13:00 entonces los únicos cuatro slots posibles son: 11:00, 11:30, 12:00 y 12:30.

Según la combinación de valores de duración de la franja y los horarios de apertura y cierre de la atracción es posible que el último slot tenga una cantidad de minutos menor a las demás y no es necesario validarlo (por ejemplo si los slots son de 40 minutos y el horario de apertura es 09:00 y el de cierre es a las 10:00, los únicos dos slots posibles son 09:00 y 09:40).

Para ello se debe implementar la clase **ParkRide** que modela a **una atracción del parque** a partir de su **nombre** (String), **horario de apertura y cierre** (instancias de `LocalTime`) y la **cantidad de minutos que tendrán los slots** (long). Si los parámetros son incorrectos se debe arrojar un error. Esta clase **debe permitir consultar todos los slots de la atracción** (indicando para cada uno de ellos el nombre de la atracción y la hora de inicio de la franja del slot) utilizando la clase **ParkSlot**. Se debe también permitir cambiar el horario de cierre de una atracción.

Deberá utilizar la clase `java.time.LocalTime` que modela horas y minutos.

Implementar ParkRide, ParkSlot y todo lo necesario para que, con el siguiente programa de prueba, se obtenga la salida indicada en los comentarios:

```
public class ParkRideTester {
    public static void main(String[] args) {
        // Ejemplo de uso de la clase java.time.LocalTime
        LocalTime ten = LocalTime.of(10,0);
        System.out.println(ten); // 10:00
        long minutesToAdd = 45;
        LocalTime tenPlus45 = ten.plusMinutes(minutesToAdd);
        System.out.println(tenPlus45); // 10:45
        LocalTime tenPlus65 = ten.plusMinutes(65);
        System.out.println(tenPlus65); // 11:05
        System.out.println(tenPlus65.isBefore(tenPlus45)); // false

        // Soarin abre a las 09:00, cierra a las 10:00 y los slots son de 15 minutos
        ParkRide soarin = new ParkRide("Soarin", LocalTime.of(9,0), LocalTime.of(10,0), 15);
        for(ParkSlot soarinSlot : soarin) {
            System.out.println(soarinSlot);
        }
        // Soarin < 09:00 Slot
        // Soarin < 09:15 Slot
        // Soarin < 09:30 Slot
        // Soarin < 09:45 Slot

        // Test Track abre a las 19:30, cierra a las 21:00 y los slots son de 60 minutos
        ParkRide tT = new ParkRide("Test Track", LocalTime.of(19,30), LocalTime.of(21,0), 60);
        for(ParkSlot tTSlot : tT) {
            System.out.println(tTSlot);
        }
        // Test Track < 19:30 Slot
        // Test Track < 20:30 Slot
```

```

Iterator<ParkSlot> soarinIt = soarin.iterator();
// Se cambia el horario de cierre de Soarin a las 09:25
soarin.setCloseTime(LocalTime.of(9,25));

while(soarinIt.hasNext()) {
    System.out.println(soarinIt.next());
}
// Soarin ◇ 09:00 Slot
// Soarin ◇ 09:15 Slot
// Soarin ◇ 09:30 Slot
// Soarin ◇ 09:45 Slot

for(ParkSlot soarinSlot : soarin) {
    System.out.println(soarinSlot);
}
// Soarin ◇ 09:00 Slot
// Soarin ◇ 09:15 Slot

try {
    // Se arroja un error porque el horario de cierre es anterior al de apertura
    new ParkRide("Spaceship Earth", LocalTime.of(9,30), LocalTime.of(7,30), 10);
} catch (Exception ex) {
    System.out.println(ex.getMessage()); // Close time cannot be before open time
}
try {
    // Se arroja un error porque la atracción debe abrir y cerrar el mismo día
    new ParkRide("Spaceship Earth", LocalTime.of(23,30), LocalTime.of(1,0), 30);
} catch (Exception ex) {
    System.out.println(ex.getMessage()); // Close time cannot be before open time
}
try {
    // Se arroja un error porque la cantidad de minutos de los slots debe ser positiva
    new ParkRide("Spaceship Earth", LocalTime.of(9,0), LocalTime.of(10,0), 0);
} catch (Exception ex) {
    System.out.println(ex.getMessage()); // Slot minutes must be positive
}
ParkRide seaBase = new ParkRide("SeaBase", LocalTime.of(9,0), LocalTime.of(9,0), 45);
try {
    seaBase.setCloseTime(LocalTime.of(8, 30));
} catch (Exception ex) {
    System.out.println(ex.getMessage()); // Close time cannot be before open time
}
try {
    // Se arroja un error pues los horarios de apertura y cierre coinciden
    System.out.println(seaBase.iterator().next());
} catch (Exception ex) {
    System.out.println(ex.getClass()); // class java.util.NoSuchElementException
}
}
}

```

Ejercicio 2 (3.5 puntos)

Se desea modelar un sistema que administre el acceso de pasajeros a los salones VIP de un aeropuerto.

Los **pasajeros** cuentan con una **cantidad limitada de pases** para acceder a los salones, donde con cada acceso se utiliza uno de los pases. La clase **Passenger** modela a un pasajero a partir de su **nombre** (String), la **aerolínea** en la que vuela (String) y la **cantidad de pases** que tiene (int).

Ya cuenta con una clase **LoungeCentral** (que no puede modificarse) que modela a la **central de todos los salones** y que cuenta con métodos **para abrir y cerrar los salones**, ambos implementados a partir de un campo que puede consultarse con el método `isOpen`.

```
public class LoungeCentral {
    private boolean isOpen = true;

    public void openLounges() {
        isOpen = true;
    }

    public void closeLounges() {
        isOpen = false;
    }

    public boolean isOpen() {
        return isOpen;
    }
}
```

Los salones deben contar con métodos para permitir el ingreso y el egreso de pasajeros a los mismos. Para el **ingreso al salón** se debe contar con un método **enter** que recibe la instancia de **Passenger**. Para el **egreso del salón** no interesa cuál fue el pasajero que egresó y se deben contar con dos métodos: uno que contemple **el egreso de un pasajero** y otro que contemple **el egreso de N pasajeros** donde N se recibe por parámetro.

Existen además tres tipos de salones:

- Un salón que permite el ingreso de pasajeros de **todas las aerolíneas** que cuenten con al menos **un pase**
- Un salón que permite el ingreso de **hasta M pasajeros** de **todas las aerolíneas** que cuenten con **al menos un pase**, donde M se indica por parámetro
- Un salón que permite el ingreso de **hasta M pasajeros** de **una aerolínea X** que cuenten con **al menos un pase**, donde M y X se indican por parámetro

Para todos los salones el ingreso de un pasajero **debe fallar si el salón está cerrado** (en función de los métodos de la central).

Completar los implementar Passenger y todo lo necesario para que, con el siguiente programa de prueba, se obtenga la salida indicada en los comentarios:

```
public class LoungeTester {
    public static void main(String[] args) {
        // Se instancia un pasajero que cuenta con 3 pases para acceder a los salones
        Passenger mark = new Passenger("Mark Smith", "Delta", 3);
        Passenger sarah = new Passenger("Sarah Johnson", "LATAM", 5);

        // Se instancia una central de salones que arranca abierta
        LoungeCentral central = new LoungeCentral();

        // Se instancia un salón "Ezeiza Lounge" que permite el ingreso de pasajeros
        // de todas las aerolíneas que cuenten con al menos un pase cuando está abierto
        ..... lounge = new .....;

        System.out.println(lounge); // Ezeiza Lounge has 0 guests

        lounge.enter(mark); // Mark Smith entra al salón utilizando un pase
        System.out.println(lounge); // Ezeiza Lounge has 1 guests
        lounge.enter(mark); // Se puede volver a ingresar al mismo salón utilizando un pase
        lounge.enter(sarah);
        System.out.println(lounge); // Ezeiza Lounge has 3 guests
    }
}
```



```

lounge.exit(); // Uno de los pasajeros egresa del salón
System.out.println(lounge); // Ezeiza Lounge has 2 guests

central.closeLounges(); // Se cierran todos los salones vinculados a la central
try {
    lounge.enter(mark); // Falla porque el salón está cerrado
} catch (Exception ex) {
    System.out.println(ex.getMessage()); // Cannot enter lounge
}
lounge.exit(); // Se puede realizar el egreso incluso con el salón cerrado
lounge.exit();
System.out.println(lounge); // Ezeiza Lounge has 0 guests
try {
    lounge.exit(); // Falla porque no hay pasajeros que puedan egresar del salón
} catch (Exception ex) {
    System.out.println(ex.getMessage()); // Lounge is empty
}

// Se instancia un salón "The Centurion" que permite el ingreso de hasta 3 pasajeros
// de todas las aerolíneas que cuenten con al menos un pase cuando está abierto
..... capLounge = new .....;

central.openLounges(); // Se abren todos los salones vinculados a la central

capLounge.enter(mark);
capLounge.enter(sarah);
try {
    capLounge.enter(mark); // Falla porque Mark Smith ya utilizó los 3 pases
} catch (Exception ex) {
    System.out.println(ex.getMessage()); // Cannot enter lounge
}
capLounge.enter(sarah);
System.out.println(capLounge); // The Centurion has 3 guests up to 3 guests
try {
    capLounge.enter(sarah); // Falla porque se alcanzó la capacidad máxima
} catch (Exception ex) {
    System.out.println(ex.getMessage()); // Cannot enter lounge
}
capLounge.exit(2); // 2 pasajeros egresan del salón
System.out.println(capLounge); // The Centurion has 1 guests up to 3 guests
try {
    capLounge.exit(2); // Falla porque no pueden egresar 2 cuando hay 1 solo
} catch (Exception ex) {
    System.out.println(ex.getMessage()); // Lounge is empty
}
System.out.println(capLounge); // The Centurion has 1 guests up to 3 guests

// Se instancia un salón "VIP Lounge" que permite el ingreso de hasta 4 pasajeros
// de la aerolínea "LATAM" que cuenten con al menos un pase cuando está abierto
..... airLounge = new .....;
airLounge.enter(sarah);
System.out.println(airLounge); // LATAM VIP Lounge has 1 guests up to 4 guests
try {
    // Falla porque no vuela con LATAM
    airLounge.enter(new Passenger("Emily Jones", "Delta", 2));
} catch (Exception ex) {
    System.out.println(ex.getMessage()); // Cannot enter lounge
}
}
}

```

Ejercicio 3 (3.5 puntos)

Se desean modelar **dos colecciones** que permitan **agregar reportes**, **consultar los reportes** agregados **a partir de su índice** y **consultar los reportes con cierto orden**. La colección **MinToMaxReport** debe permitir consultar los reportes con orden ascendente. La colección **MaxToMinReport** debe permitir consultar los reportes con orden descendente. Ambas colecciones **reciben el criterio de orden al momento de instanciarla**.

Ya cuenta con la interfaz **ReportCollection** (que puede modificarse):

```
public interface ReportCollection<R> {

    void add(R report);

    R get(int index);

    R[] reports();

}
```

Completar los , implementar MinToMaxReport, MaxToMinReport y todo lo necesario para que, con el siguiente programa de prueba, se obtenga la salida indicada en los comentarios:

```
public class ReportTester {
    public static void main(String[] args) {
        // Se instancia un reporte de palabras MINToMax
        // indicando en el constructor un orden alfabético
        Comparator<String> alphaCmp = .....

        ReportCollection<String> minToMaxWords = new MinToMaxReport<>(alphaCmp);

        minToMaxWords.add("Foo");
        minToMaxWords.add("Bar");
        minToMaxWords.add("Foo");

        // Se obtiene la primer palabra reportada (por orden de inserción)
        System.out.println(minToMaxWords.get(0)); // Foo
        // Se obtienen todas las palabras en orden ascendente
        // en función del orden indicado en el constructor
        System.out.println(Arrays.toString(minToMaxWords.reports()));
        // [Bar, Foo, Foo]

        // Se instancia un reporte de palabras MAXToMin
        // indicando en el constructor un orden alfabético
        ReportCollection<String> maxToMinWords = new MaxToMinReport<>(alphaCmp);

        maxToMinWords.add("Foo");
        maxToMinWords.add("Bar");
        maxToMinWords.add("Foo");

        System.out.println(maxToMinWords.get(2)); // Foo
        // Se obtienen todas las palabras en orden descendente
        // en función del orden indicado en el constructor
        System.out.println(Arrays.toString(maxToMinWords.reports()));
        // [Foo, Foo, Bar]
```





```

try {
    // Se arroja un error porque no se reportó una cuarta palabra
    maxToMinWords.get(3);
} catch (IllegalArgumentException ex) {
    System.out.println("Error"); // Error
}

// Se instancia un reporte de temperaturas MINToMax indicando en el constructor
// un orden descendente por temperatura y desempata alfabético por ciudad
Comparator<Weather> weatherCmp = .....

ReportCollection<Weather> minToMaxTemps = new MinToMaxReport<>(weatherCmp);

minToMaxTemps.add(new Weather("Juneau", 15));
minToMaxTemps.add(new Weather("Juneau", 20));
minToMaxTemps.add(new Weather("Atlanta", 15));

// Se obtienen todos las temperaturas en orden ascendente
// en función del orden indicado en el constructor
System.out.println(Arrays.toString(minToMaxTemps.reports()));
// [It's 20 degrees in Juneau, It's 15 degrees in Atlanta, It's 15 degrees in Juneau]

// Se instancia un reporte de térmicas MAXToMin indicando en el constructor
// un orden descendente por temperatura y desempata alfabético por ciudad
ReportCollection<FeelsLikeWeather> maxToMinSts = new MaxToMinReport<>(weatherCmp);

maxToMinSts.add(new FeelsLikeWeather("Juneau", 15, 16));
maxToMinSts.add(new FeelsLikeWeather("Juneau", 20, 20));
maxToMinSts.add(new FeelsLikeWeather("Atlanta", 15, 13));

// Se obtienen todos las térmicas en orden descendente
// en función del orden indicado en el constructor
System.out.println(Arrays.toString(maxToMinSts.reports()));
// [Feels like 16 degrees, Feels like 13 degrees, Feels like 20 degrees]
}

static class Weather {
    private final String city;
    private final int temperature;
    public Weather(String city, int temperature) {
        this.city = city;
        this.temperature = temperature;
    }
    @Override
    public String toString() {
        return "It's %d degrees in %s".formatted(temperature, city);
    }
}

static class FeelsLikeWeather extends Weather {
    private final int feelsLike;
    public FeelsLikeWeather(String city, int temperature, int feelsLike) {
        super(city, temperature);
        this.feelsLike = feelsLike;
    }
    @Override
    public String toString() {
        return "Feels like %d degrees".formatted(feelsLike);
    }
}
}

```